

5. Arquitectura de Software del Proyecto echo

La arquitectura de software de **echo** está diseñada como un sistema embebido de tiempo real orientado a eventos. Utiliza de forma asíncrona los dos núcleos independientes del microcontrolador RP2040 (Raspberry Pi Pico) mediante un modelo estricto de **Polling + Interrupciones (IRQ)**, coordinado por máquinas de estado finitas (FSM) y banderas (*flags*) atómicas en memoria compartida.

A. Subsistemas y Librerías

El firmware del sistema se divide en dos grandes sub-sistemas lógicos asignados a núcleos dedicados para garantizar la ejecución determinista del muestreo y la inmunidad al *jitter*:

1. Subsistema de Abstracción de Hardware y Control (Asignado al CORE 0)

- **Misión:** Orquestar de forma directa los periféricos de entrada y salida. Maneja la temporización base, el control síncrono del multiplexor, la lectura inercial por I2C y el despacho de comandos de audio por UART.
- **Librerías del Pico SDK utilizadas (C Puro):**
 - `<hardware/adc.h>` y `<hardware/dma.h>`: Control de la digitalización automatizada.
 - `<hardware/pio.h>`: Interfaz con las máquinas de estado de I/O programable.
 - `<hardware/i2c.h>` y `<hardware/uart.h>`: Comunicación digital con la IMU y los actuadores.
 - `<pico/multicore.h>`: Sincronización y arranque seguro del segundo núcleo.

2. Subsistema de Procesamiento de Señales e Inferencia de IA (Asignado al CORE 1)

- **Misión:** Ejecutar de forma asíncrona las tareas de cómputo matemático pesado. Realiza el filtrado digital por media móvil y resuelve la clasificación del gesto mediante un modelo jerárquico.
- **Librerías utilizadas:** Código nativo en **C puro** desarrollado a medida, diseñado para invocar directamente las rutinas de punto flotante (float) altamente optimizadas grabadas en la memoria ROM del RP2040. Esto evita dependencias pesadas de C++ (como TensorFlow Lite for Microcontrollers), garantizando que el firmware ocupe una fracción mínima de los 264 KB de SRAM.

B. Identificación de Interrupciones (IRQ) y Mecanismos de Sincronización

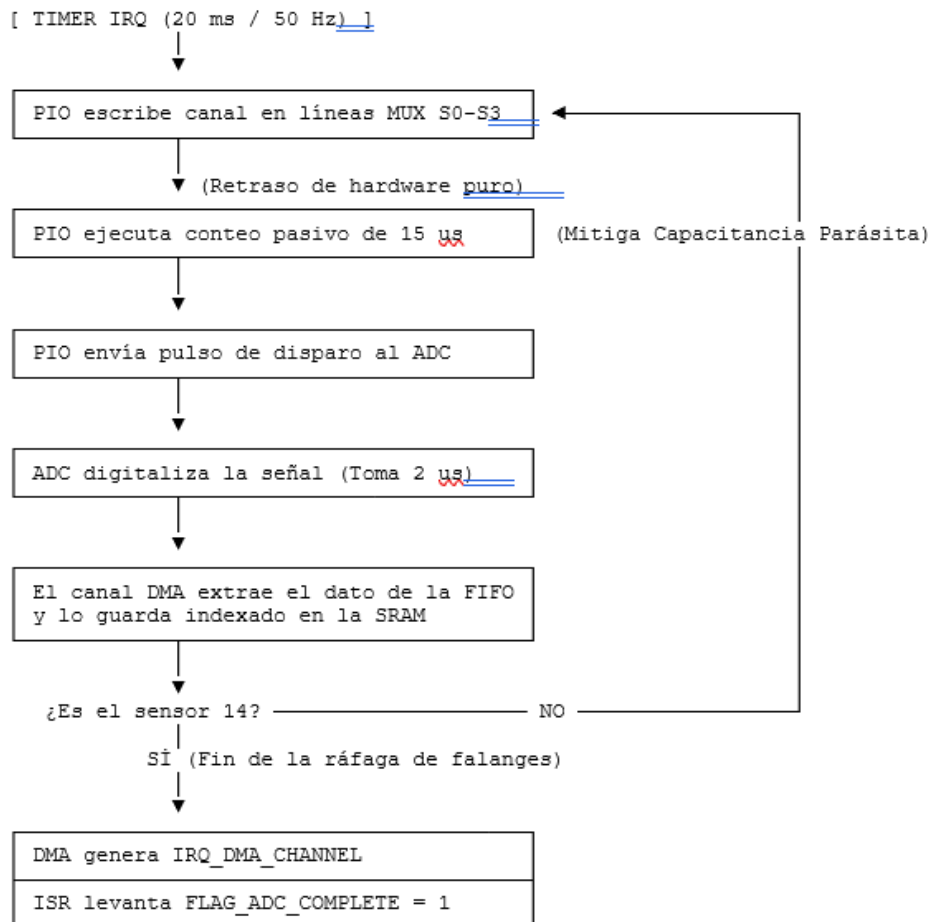
El flujo secuencial se elimina por completo; el procesador permanece en estado de bajo consumo o consulta pasiva (*polling* de banderas) hasta que las siguientes interrupciones de hardware fuerzan la ejecución de las tareas:

- **IRQ_HARDWARE_TIMER:** Configurada mediante el módulo de temporizadores del RP2040 a una frecuencia exacta de **50 Hz** (período de 20 ms). Su rutina de servicio (ISR) despierta determinísticamente a la máquina de adquisición física.

- **IRQ_DMA_CHANNEL:** Se dispara por hardware de forma autónoma en el instante exacto en que el canal DMA termina de transferir la muestra número 14 desde la FIFO del ADC a la memoria RAM. Su ISR ejecuta una única acción de software: activar la bandera atómica `FLAG_ADC_COMPLETE = 1`.
- **Sincronización Inter-Núcleo:** El Core 0 actúa como *Productor* de datos físicos y el Core 1 como *Consumidor* inteligente. La transferencia segura del vector de 20 variables se sincroniza mediante las colas físicas de silicio del chip (Hardware SIO FIFO) y la bandera global `FLAG_DATA_READY`.

C. Pipeline de Adquisición de Alta Velocidad (Hardware Autónomo)

Para esquivar por completo el efecto de "arrastre" de voltaje provocado por la capacitancia parásita de 50 pF del multiplexor CD74HC4067, se implementa un pipeline gobernado por los co-procesadores de I/O programable (PIO) y el DMA, liberando al Core 0 de ejecutar retrasos bloqueantes (`delay_us`):



Métrica de Rendimiento Absoluto: Todo este proceso toma exactamente $17\mu\text{s} * 14 = 238\mu\text{s}$ (0.238 ms). El hardware resuelve la adquisición consumiendo apenas el **1.19%** del tiempo disponible en el ciclo de 20 ms, dejando la CPU libre.

D. Máquinas de Estado del Software (FSM)

El comportamiento lógico del firmware está segmentado en dos máquinas de estado concurrentes que corren en bucles de polling orientados a eventos de bandera:

FSM del Core 0 (Adquisición, Sincronización y Salida)

1. **ESTADO_INIT:** Configura el reloj del sistema, inicializa los canales analógicos, el bus I2C de la IMU, los periféricos UART0/UART1, arranca el programa del PIO y despierta al Core 1. Transita automáticamente a **ESTADO_IDLE**.
2. **ESTADO_IDLE:** El lazo principal realiza *polling* pasivo esperando el disparo de la interrupción del temporizador. Al completarse la ráfaga autónoma del DMA (`FLAG_ADC_COMPLETE == 1`), transita a **ESTADO_READ_IMU**.
3. **ESTADO_READ_IMU:** Lee los 6 ejes (acelerómetro y giroscopio) de la IMU MPU-6050 vía I2C. Empaqueta el vector completo de 20 floats (14 Hall + 6 IMU) en el buffer de intercambio, levanta `FLAG_DATA_READY = 1` para despertar al Core 1, limpia las banderas de adquisición y transita a **ESTADO_POLL_RESPONSE**.
4. **ESTADO_POLL_RESPONSE:** Monitorea si el Core 1 depositó un ID de gesto válido en la FIFO inter-núcleo. Si se detecta, despacha de inmediato el comando de audio por el bus UART0 al DFPlayer Mini y la cadena de texto por el bus UART1 al PC de respaldo. Retorna a **ESTADO_IDLE**.

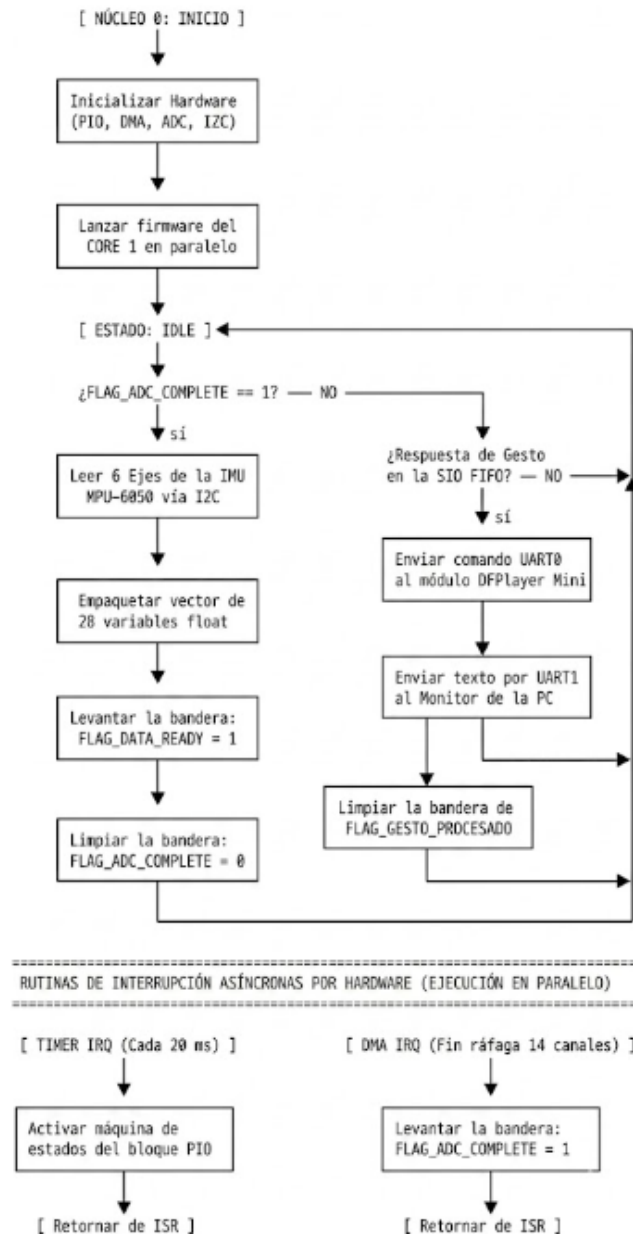
FSM del Core 1 (Procesamiento y Clasificador Jerárquico)

1. **ESTADO_WAIT_DATA:** Permanece en un lazo de *polling* de ultra baja latencia evaluando el estado de la bandera compartida: `if (FLAG_DATA_READY == 1)`. Al ser verdadera, copia el vector a su memoria local y transita a **ESTADO_DSP_FILTER**.
2. **ESTADO_DSP_FILTER:** Procesa las 14 entradas analógicas de los sensores Hall aplicando un algoritmo de filtro digital de media móvil para remover el ruido electromagnético de las líneas del guante. Transita a **ESTADO_VAR_GATE**.
3. **ESTADO_VAR_GATE:** Calcula el diferencial y la varianza temporal de los 6 ejes de la IMU para determinar si el usuario está realizando un movimiento espacial:
 - **Si Varianza < Umbral (Mano Estática):** Redirige el flujo hacia la **MLP Estática** (Inferencia optimizada en C puro para las 14 entradas de los dedos, procesando entre 16 y 24 neuronas ocultas para resolver las letras estáticas del abecedario).
 - **Si Varianza > Umbral (Mano Dinámica):** Redirige el flujo hacia la **MLP Dinámica** (Inferencia densa de 20 entradas para evaluar trayectorias y resolver frases completas como "Hola").
4. **ESTADO_CONFIRMATION:** Evalúa la salida probabilística del modelo mediante la función Softmax. Si el índice de la letra o frase seleccionada sostiene una certeza $> 90\%$ durante 3 ciclos de muestreo consecutivos (filtro de antirrebote para evitar falsos positivos), inyecta el ID del gesto en la FIFO de hardware para alertar al Core 0. Baja la bandera `FLAG_DATA_READY = 0` y regresa a **ESTADO_WAIT_DATA**.

E. Diagramas de Flujo Particionados del Algoritmo

A continuación, se presentan de forma particionada y detallada los diagramas de flujo lógicos que gobiernan de manera paralela ambos núcleos de procesamiento del proyecto **echo**:

1. Diagrama de Flujo del Lazo Principal y Rutinas de Interrupción (CORE 0)



2. Diagrama de Flujo del Procesamiento de IA e Inferencia (CORE 1)

